Практическое задание № 1. Внимание своими руками

Мультимодальная обработка данных • Вариант 42

1. Реализация четырёх функций

split_heads сначала делит каналы на головы, затем переставляет оси. merge_heads выполняет обратную перестановку и делает тензор непрерывным перед view. Внимание вычисляет softmax(QK^T / √d_k)V: маска применяется до softmax, нормировка идёт по ключам, dropout - по полученным весам. Блок использует pre-norm и две остаточные связи.

В TODO 3 отдельно пояснён порядок маскирования: обнуление весов после softmax без повторной нормировки уменьшает сумму строки. Замена запрещённых логитов на -inf до softmax сразу даёт правильное распределение по разрешённым ключам.

2. Найденные ошибки

Строки указаны внутри исходной ячейки с определениями модели; строка 1 - from __future__ import annotations.

Место	Что нарушено	Исправление
Позиционное кодирование Строка 39	Позиции начинаются с 1. Первая строка не соответствует позиции 0: синусы не нулевые, косинусы не единичные. Это нарушение заданной формулы, а не самостоятельное доказательство невозможности обучения.	arange(1, length + 1) → arange(length)
LayerNorm.forward Строки 60, 61	Статистики считаются по батчу. Выход зависит от других примеров; при B=1 нормализованная часть равна нулю. Требуется отдельная нормировка признаков каждого токена.	dim=0 → dim=-1 в mean и var; unbiased=False сохранён.
Внимание Строка 101	Softmax нормирует столбцы по запросам. Строки перестают быть распределениями по ключам. При причинной маске знаменатели могут зависеть от будущих запросов.	softmax(scores, dim=-2) → dim=-1.

В первом ноутбуке - 17 из 17 проверок. Во втором: 7/17 до исправлений, 9/17 после исправления позиций, 12/17 после LayerNorm, 17/17 после softmax. Проверки преподавателя не изменялись.

Ошибки могут скрывать друг друга: тест отсутствия доступа к будущему падает после исправления LayerNorm, когда вход перестаёт обнуляться при B=1. Поэтому каждое исправление проверялось повторным прогоном всего набора.

3. Обучение и результат на CPU

Корпус - повторение фразы «алгоритм внимания смотрит на все позиции сразу. » 200 раз. Посимвольный словарь: 19 символов. Модель: 2 блока, 4 головы, d_model=128, d_ff=512, контекст 64, dropout=0.1; 400,640 параметров.

Обучение: 600 шагов, батч 32, AdamW, lr=0.0003, weight_decay=0.01. Seed обучения 0; seed варианта ошибок 42. Генерация: seed 0, начало «а», 200 новых символов, temperature=0.8, top_k=10.

Потери первого и последнего обучающих батчей: **3.184385** → **0.036880**.



Пример генерации

ания смотрит на все позиции сразу. алгоритм внимания смотрит на все позиции сразу. алгоритм внимания смотрит на все позиции сразу. алгоритм внимания смотрит на все позиции сразу. алгоритм внимания смот

Модель воспроизводит структуру учебной фразы. Это показывает способность обучаться на заданных данных. Отдельной валидации здесь нет, поэтому по этому эксперименту нельзя судить об обобщении на новые тексты. Потери сняты с обучающих батчей при включённом dropout.

4. Воспроизводимость

Сохранённый запуск: Windows, Python 3.13.1, PyTorch 2.10.0+cpu, Matplotlib 3.10.7, CPU, 2 потока. Оба ноутбука выполнены сверху вниз в чистых ядрах. Дополнительно оба ноутбука выполнены в Google Colab с Tesla T4: 17/17 проверок в каждом, обучение второго ноутбука на CUDA завершило 600 шагов. Потери GPU-запуска: 3.190 → 0.034 (округлено по выводу).

Окружение Colab: Python 3.13.16, PyTorch 2.11.0+cu130, Matplotlib 3.10.0. Выполненные ноутбуки, вывод устройства, график и текст GPU-запуска находятся в practical_assignment_1/colab. Штатные тесты используют CPU; обучение в Colab использует T4.

Инструкция запуска находится в README.md. Журналы проверок, параметры эксперимента, все 600 значений потерь, график и текст сохранены в practical_assignment_1/results. Между устройствами и версиями библиотек возможны численные различия.

Основа: выданные ноутбуки практического задания № 1; занятия 1.2-1.4 и вебинары 3-8. В PDF условия приведён другой корпус, поэтому его значения потерь не использовались как точный порог.